Proyecto Final (Lavadora)

Mario Carrasco 2022-1510

Cristian Jiménez 2024-0712

Resumen- Este es el proyecto final, es una Lavadora automática que va por ciclos, se presentará la teoría, como los diagramas y como funcionará, el proceso y la lógica del código de como ira montada y la conclusión del físico.

Abstract- This is the final project; it's an automatic washing machine that works in cycles. The theory will be presented, along with diagrams showing how it will function, the process and logic of the code for how it will be assembled, and the physical conclusion.

I. INTRODUCCIÓN

En la electrónica digital, es posible controlar procesos automáticos usando circuitos programables como las FPGA. Uno de los métodos más utilizados para esto es la máquina de estados, que permite organizar un sistema en diferentes etapas y controlar cómo cambia de una a otra.

En este trabajo se diseñó una lavadora automática utilizando el FPGA. El sistema controla las distintas etapas del lavado, como llenado, lavado, enjuague y centrifugado, además de responder a botones del usuario como encendido, pausa y reinicio.

II. MARCO TEORICO

Para entender el funcionamiento de este proyecto hay que saber exactamente que se quiere:

Un sistema de una lavadora el cual funcione por estados: IDLE, FILL1, WASH, DRAIN1, FILL2, RINSE, DRAIN2, SPIN, PAUSE.

Con estos estados podemos entender que al presionar el botón de START la lavadora debe pasar por esos estados para completar el ciclo.

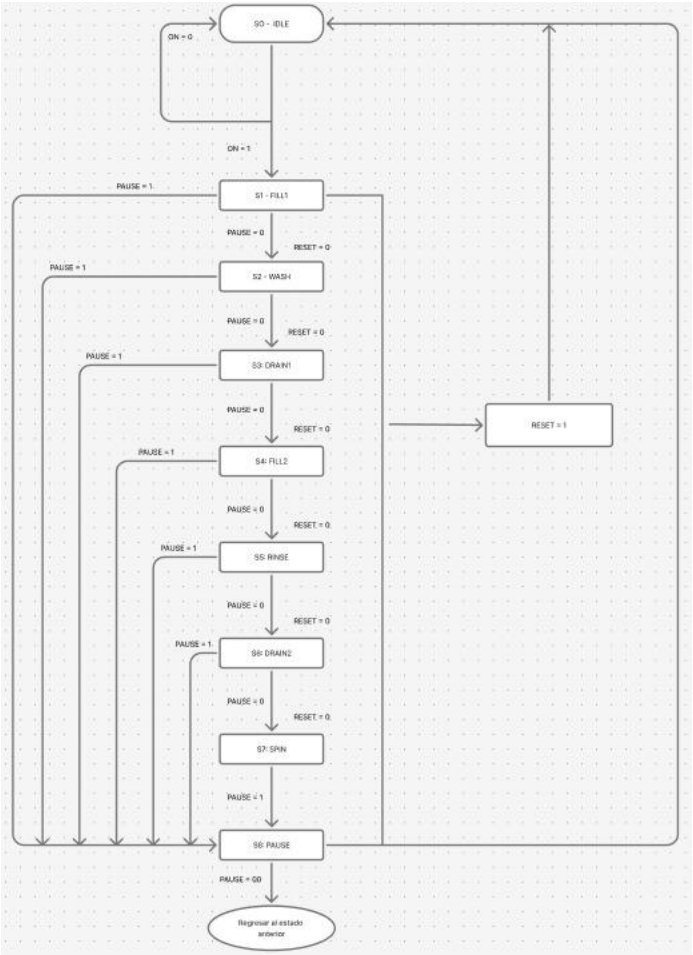


Diagrama de Estados

Estado actual	Entrada ON	Entrada SKIP	Próximo estado
IDLE	1	X	FILL1
FILL1	X	1	WASH
WASH	X	1	DRAIN1
DRAIN1	X	1	FILL2
FILL2	X	1	RINSE
RINSE	X	1	DRAIN2
DRAIN2	X	1	SPIN
SPIN	X	1	IDLE
PAUSE	X	X	after_pause

Tabla de transiciones

Con esta tabla podremos determinar qué estado será el siguiente dependiendo de la entrada que le demos.

Las entradas que tenemos en el sistema son 4 botones y 1 motor:

Boton ON: para encender la lavadora e iniciar el ciclo.

Boton RESET: para detener el ciclo y volver a empezar.

Boton SKIP: para saltar al siguiente estado o al siguiente paso del ciclo.

Boton PAUSE: para pausar el estado actual o ciclo de la lavadora y mantenerlo.

Motor: esto simula el giro de la lavadora.

III. MONTAJE DEL LABORATORIO

Explicación del funcionamiento del código:

El código implementa una máquina de estados finitos que controla el funcionamiento de la lavadora de manera secuencial. Al encender la FPGA, el sistema inicia en un estado de reposo llamado IDLE, en el cual todas las salidas permanecen apagadas. En este estado, la lavadora espera a que el usuario presione el botón de encendido para comenzar el ciclo de lavado.

Cuando se presiona el botón ON, el sistema pasa al estado de llenado inicial (FILL1). En esta etapa se activa la bomba de llenado para permitir la entrada de agua durante un tiempo determinado. Una vez transcurrido ese tiempo, el sistema no cambia directamente a la siguiente etapa, sino que entra en un estado de pausa que sirve como transición, donde todas las salidas se apagan por unos segundos.

Después de la pausa, el sistema entra en la etapa de lavado (WASH). En este estado se activa el motor principal, pero no inmediatamente, ya que primero permanece apagado durante un segundo y luego se enciende. Además, el inversor de giro cambia periódicamente entre 0 y 1, lo que permite simular el movimiento alternado del tambor de la lavadora. Al finalizar el tiempo de lavado, el sistema vuelve a entrar en una pausa antes de continuar.

A continuación, el sistema pasa a la etapa de drenado (DRAIN1), donde se activa la bomba de salida de agua para vaciar el tambor. Una vez completado este proceso, se introduce nuevamente una pausa antes de continuar con el segundo llenado (FILL2), en el cual se repite el proceso de entrada de agua controlado por tiempo.

Posteriormente, el sistema realiza el enjuague (RINSE), que funciona de manera similar al lavado, activando el motor con cambio de dirección. Tras finalizar esta etapa, se realiza otro drenado (DRAIN2) para eliminar el agua restante, seguido nuevamente por una pausa de transición.

Finalmente, el sistema entra en la etapa de centrifugado (SPIN). En este estado, el motor se activa después de un segundo y permanece girando en una sola dirección, ya que el inversor de giro se mantiene en un valor fijo. Al concluir el tiempo de centrifugado, el sistema regresa al estado inicial IDLE y todas las salidas se apagan, indicando el fin del ciclo de lavado.

El comportamiento de cada una de estas etapas está controlado por parámetros que definen su duración. Estos parámetros establecen el tiempo que el sistema permanece en cada estado, como el llenado, lavado, drenado, enjuague y centrifugado, así como el tiempo de pausa entre etapas. Además, existe un parámetro que define cada cuánto tiempo cambia el sentido de giro del motor durante el lavado y el enjuague. Estos valores permiten ajustar el funcionamiento de la lavadora sin modificar la lógica principal del sistema.

El control del sistema se completa con las entradas del usuario. El botón de encendido permite iniciar el ciclo desde el estado de reposo, el botón de reinicio detiene todo el proceso y regresa al estado inicial, el botón de salto permite avanzar manualmente a la siguiente etapa y el botón de pausa congela temporalmente el sistema sin perder el estado actual. De esta manera, el código logra un control completo y ordenado del proceso de lavado utilizando una estructura secuencial basada en estados.

Se anexara el codigo a continuación:

```
module debounce #(
    parameter integer COUNT_MAX = 540000
)(
    input  wire clk,
    input  wire btn_in,
```

```
    output reg  btn_out = 1'b1
);

    reg [19:0] counter = 0;
    reg sync_0 = 1'b1;
    reg sync_1 = 1'b1;
    reg stable_state = 1'b1;

    always @(posedge clk) begin
        sync_0 <= btn_in;
        sync_1 <= sync_0;

        if (sync_1 != stable_state) begin
            if (counter < COUNT_MAX)
                counter <= counter + 1'b1;
            else begin
                stable_state <= sync_1;
                btn_out <= sync_1;
                counter <= 0;
            end
        end else begin
            counter <= 0;
        end
    end

endmodule

module one_pulse(
    input  wire clk,
    input  wire signal_in,
    output wire pulse_out
);

    reg signal_d = 1'b0;

    always @(posedge clk) begin
        signal_d <= signal_in;
    end

    assign pulse_out = signal_in & ~signal_d;

endmodule

module sec_tick #(
    parameter integer CLK_HZ = 27000000
)(
    input  wire clk,
    input  wire rst,
    input  wire en,
    output reg  tick = 1'b0
);

    localparam integer MAX_COUNT = CLK_HZ -
1;
    reg [31:0] counter = 0;

    always @(posedge clk) begin
        if (rst) begin
            counter <= 0;
            tick <= 1'b0;
        end else if (en) begin
            if (counter >= MAX_COUNT) begin
                counter <= 0;
                tick <= 1'b1;
            end else begin
                counter <= counter + 1'b1;
                tick <= 1'b0;
            end
        end else begin
            counter <= 0;
```

```
            tick <= 1'b0;
        end
    end

endmodule

module lavadora_top #(
    parameter integer CLK_HZ       = 27000000,
    parameter integer T_FILL_SEC   = 40,
    parameter integer T_WASH_SEC   = 40,
    parameter integer T_DRAIN_SEC  = 40,
    parameter integer T_RINSE_SEC  = 40,
    parameter integer T_SPIN_SEC   = 40,
    parameter integer T_PAUSE_SEC  = 4,
    parameter integer T_TOGGLE_SEC = 2
)(
    input  wire clk,
    input  wire btn_on,
    input  wire btn_reset,
    input  wire btn_skip,
    input  wire btn_pause,

    output reg  [3:0] leds,
    output reg  dir_inv,
    output reg  motor_en,
    output reg  pump_fill,
    output reg  pump_drain,
    output reg  on_led
);

    wire on_db_n, reset_db_n, skip_db_n, pause_db_n;

    debounce d0 (.clk(clk), .btn_in(btn_on), .btn_out(on_db_n));
    debounce d1 (.clk(clk), .btn_in(btn_reset), .btn_out(reset_db_n));
    debounce d2 (.clk(clk), .btn_in(btn_skip), .btn_out(skip_db_n));
    debounce d3 (.clk(clk), .btn_in(btn_pause), .btn_out(pause_db_n));

    wire on_db    = ~on_db_n;
    wire reset_db = ~reset_db_n;
    wire skip_db  = ~skip_db_n;
    wire pause_db = ~pause_db_n;

    wire on_pulse, reset_pulse, skip_pulse, pause_pulse;

    one_pulse p0 (.clk(clk), .signal_in(on_db), .pulse_out(on_pulse));
    one_pulse p1 (.clk(clk), .signal_in(reset_db), .pulse_out(reset_pulse));
    one_pulse p2 (.clk(clk), .signal_in(skip_db), .pulse_out(skip_pulse));
    one_pulse p3 (.clk(clk), .signal_in(pause_db), .pulse_out(pause_pulse));

    wire sec_pulse;
    reg paused = 1'b0;

    sec_tick #(
        .CLK_HZ(CLK_HZ)
    ) u_tick (
        .clk(clk),
        .rst(reset_pulse),
        .en(~paused),
        .tick(sec_pulse)
    );

    localparam [3:0]
        S_IDLE    = 4'd0,
```

```
    S_FILL1   = 4'd1,
    S_WASH    = 4'd2,
    S_DRAIN1  = 4'd3,
    S_FILL2   = 4'd4,
    S_RINSE   = 4'd5,
    S_DRAIN2  = 4'd6,
    S_SPIN    = 4'd7,
    S_PAUSE   = 4'd8;

reg [3:0] state = S_IDLE;
reg [3:0] after_pause_state = S_IDLE;

reg [15:0] sec_count = 0;
reg [15:0] motor_delay = 0;
reg [15:0] dir_count = 0;

always @(posedge clk) begin
    if (reset_pulse)
        paused <= 1'b0;
    else if (pause_pulse && state != S_IDLE)
        paused <= ~paused;
end

always @(posedge clk) begin
    if (reset_pulse) begin
        state <= S_IDLE;
        after_pause_state <= S_IDLE;
        sec_count <= 0;
        motor_delay <= 0;
        dir_count <= 0;
        dir_inv <= 1'b0;
        motor_en <= 1'b0;
        pump_fill <= 1'b0;
        pump_drain <= 1'b0;
        on_led <= 1'b0;
    end else if (!paused) begin

        if (state == S_IDLE && on_pulse) begin
            state <= S_FILL1;
            on_led <= 1'b1;
            sec_count <= 0;
            motor_delay <= 0;
            dir_count <= 0;
        end

        else if (skip_pulse && state != S_IDLE)
begin
            case (state)
                S_FILL1: begin state <= S_PAUSE;
after_pause_state <= S_WASH; end
                S_WASH:  begin state <= S_PAUSE;
after_pause_state <= S_DRAIN1; end
                S_DRAIN1:begin state <= S_PAUSE;
after_pause_state <= S_FILL2; end
                S_FILL2: begin state <= S_PAUSE;
after_pause_state <= S_RINSE; end
                S_RINSE: begin state <= S_PAUSE;
after_pause_state <= S_DRAIN2; end
                S_DRAIN2:begin state <= S_PAUSE;
after_pause_state <= S_SPIN; end
                S_SPIN:  begin state <= S_IDLE;
after_pause_state <= S_IDLE; on_led <= 1'b0; end
                S_PAUSE: begin state <=
after_pause_state; end
                default: begin state <= S_IDLE;
after_pause_state <= S_IDLE; end
            endcase

            sec_count <= 0;
            motor_delay <= 0;
            dir_count <= 0;
            dir_inv <= 1'b0;
            motor_en <= 1'b0;
            pump_fill <= 1'b0;
```

```
        pump_drain <= 1'b0;
    end

    else if (sec_pulse) begin
        case (state)
            S_IDLE: begin
                sec_count <= 0;
                motor_en <= 1'b0;
                pump_fill <= 1'b0;
                pump_drain <= 1'b0;
            end

            S_FILL1: begin
                pump_fill <= 1'b1;
                if (sec_count >= T_FILL_SEC - 1)
begin
                    state <= S_PAUSE;
                    after_pause_state <= S_WASH;
                    sec_count <= 0;
                end else sec_count <= sec_count +
1'b1;
            end

            S_WASH: begin
                if (motor_delay >= 1) motor_en <=
1'b1;
                else motor_delay <= motor_delay +
1'b1;

                if (dir_count >= T_TOGGLE_SEC -
1) begin
                    dir_inv <= ~dir_inv;
                    dir_count <= 0;
                end else dir_count <= dir_count +
1'b1;

                if (sec_count >= T_WASH_SEC - 1)
begin
                    state <= S_PAUSE;
                    after_pause_state <= S_DRAIN1;
                    sec_count <= 0;
                    motor_delay <= 0;
                end else sec_count <= sec_count +
1'b1;
            end

            S_DRAIN1: begin
                pump_drain <= 1'b1;
                if (sec_count >= T_DRAIN_SEC - 1)
begin
                    state <= S_PAUSE;
                    after_pause_state <= S_FILL2;
                    sec_count <= 0;
                end else sec_count <= sec_count +
1'b1;
            end

            S_FILL2: begin
                pump_fill <= 1'b1;
                if (sec_count >= T_FILL_SEC - 1)
begin
                    state <= S_PAUSE;
                    after_pause_state <= S_RINSE;
                    sec_count <= 0;
                end else sec_count <= sec_count +
1'b1;
            end

            S_RINSE: begin
                if (motor_delay >= 1) motor_en <=
1'b1;
                else motor_delay <= motor_delay +
1'b1;
```

```
            if (dir_count >= T_TOGGLE_SEC -
1) begin
                dir_inv <= ~dir_inv;
                dir_count <= 0;
            end else dir_count <= dir_count +
1'b1;

            if (sec_count >= T_RINSE_SEC - 1)
begin
                state <= S_PAUSE;
                after_pause_state <= S_DRAIN2;
                sec_count <= 0;
                motor_delay <= 0;
            end else sec_count <= sec_count +
1'b1;
        end

        S_DRAIN2: begin
            pump_drain <= 1'b1;
            if (sec_count >= T_DRAIN_SEC - 1)
begin
                state <= S_PAUSE;
                after_pause_state <= S_SPIN;
                sec_count <= 0;
            end else sec_count <= sec_count +
1'b1;
        end

        S_SPIN: begin
            dir_inv <= 1'b0;
            if (motor_delay >= 1) motor_en <=
1'b1;
            else motor_delay <= motor_delay +
1'b1;

            if (sec_count >= T_SPIN_SEC - 1)
begin
                state <= S_IDLE;
                on_led <= 1'b0;
                sec_count <= 0;
            end else sec_count <= sec_count +
1'b1;
        end

        S_PAUSE: begin
            if (sec_count >= T_PAUSE_SEC - 1)
begin
                state <= after_pause_state;
                sec_count <= 0;
            end else sec_count <= sec_count +
1'b1;
        end
      endcase
    end
  end

  always @(*) begin
    case (state)
      S_IDLE:   leds = 4'b0000;
      S_FILL1:  leds = 4'b0001;
      S_WASH:   leds = 4'b0010;
      S_DRAIN1: leds = 4'b0011;
      S_FILL2:  leds = 4'b0100;
      S_RINSE:  leds = 4'b0101;
      S_DRAIN2: leds = 4'b0110;
      S_SPIN:   leds = 4'b0111;
      S_PAUSE:  leds = 4'b1000;
      default:  leds = 4'b0000;
    endcase
  end

endmodule
```

Funcionamiento:

Al presionar el boton ON saldra del ciclo de espera y empezara el primer ciclo que es FILL1 y luego del tiempo establecido ira cambiando de estado automaticamente, siguiento el estado de WASH, luego el estado de DRAIN1, seguido de FILL2, RINSE, DRAIN2 y por ultimo SPIN. El estado de PAUSE solo aparecera si se presiona el boton de PAUSE, el SKIP saltara al siguiente estado, por ejemplo si esta en el estado de WASH y se le da al boton de SKIP entonces sin tiempo de espera pasara al estado de DRAIN1. Por ultimo el RESET reiniciara por complete todos los estados.

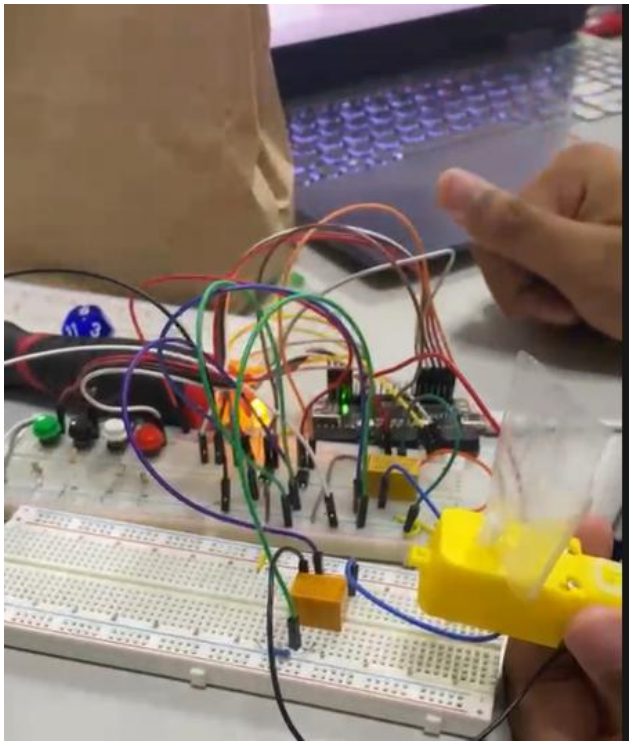


Imagen del circuito montado

https://drive.google.com/file/d/1ViyBTmKFT6Y39VRUJlzGH0bI6I_xvm3c/view?usp=drivesdk

IV. CONCLUSIÓN

En este trabajo se logró diseñar e implementar una lavadora automatizada utilizando una FPGA, basada en una máquina de estados finitos que controla de forma ordenada cada etapa del proceso de lavado. El sistema integra el uso de temporización, control de entradas mediante botones y manejo de múltiples salidas como motores y bombas, demostrando la aplicación práctica de conceptos de lógica digital. Como resultado, se obtuvo un sistema funcional, modular y fácil de ajustar, evidenciando la utilidad de las FPGA en el desarrollo de soluciones de control automático.